

Claude Code in 90 Tagen — ein kostenloser Selbstlernkurs

Lerne, das Maximum aus Claude Code herauszuholen — vom ersten Befehl bis zu eigenen Agenten und Automatisierung. 80% Praxis, 20% Theorie. Ohne Zertifikat, dafür mit etwas Besserem: 6 fertigen Mini-Projekten.

Für wen: für alle, die die Grundlagen des Terminals kennen (Terminal öffnen, mit `cd` in einen Ordner wechseln) und lernen wollen, mit einem KI-Agenten Software zu bauen. Du musst kein Profi-Entwickler sein — Claude schreibt den Code, du lernst, ihn zu führen.

Voraussetzungen:

- **Claude Code** installiert (CLI oder Desktop-App) und ein Claude-Konto
- **Git** installiert (wird erst ab Modul 6 gebraucht, aber installiere es gleich)
- Beliebiges Betriebssystem (Windows / Mac / Linux)

So nutzt du den Kurs:

- 12 Wochen, 6 Module (jedes Modul = 2 Wochen)
- Rhythmus: 5 Tage × 45–60 Minuten Übung + 1 Tag Retrospektive (was habe ich gelernt, was ist noch unklar)
- Jedes Modul baut ein **Mini-Projekt von Null** — du brauchst keinerlei vorhandenen Code
- Die Übungen sind mit **Ü** markiert (z.B. Ü1.3 = Modul 1, Übung 3). Zu jeder Übung gibt es am Ende des Kurses eine Lösung im Abschnitt **Lösungen** — aber **versuche es zuerst selbst**. Fehler gehören zum Lernen; schau die Lösung erst an, wenn du feststeckst.
- Jedes Modul endet mit einem **Wissenscheck** — einer Aufgabe ohne Anleitung, nur mit einem Ziel. Das ist deine "Prüfung" statt eines Zertifikats.

Modulübersicht:

Modul	Thema	Wochen	Mini-Projekt
1	Claude-CLI-Grundlagen	1–2	Ausgaben-Tagebuch (Python-Konsolenanwendung)
2	CLAUDE.md und Gedächtnis	3–4	Persönliche Portfolio-Seite (HTML/CSS/JS)
3	Plan-Modus und Loop	5–6	Quiz-App (Web)
4	MCP-Server	7–8	Wetter + Notizen (externe Daten + Datenbank)
5	Parallele Subagenten	9–10	Analyse eines Open-Source-Projekts
6	Git-Integration und Automatisierung	11–12	Abschlussprojekt nach Wahl

Modul 1 — Claude-CLI-Grundlagen (Wochen 1–2)

Mini-Projekt: "Ausgaben-Tagebuch" — eine Python-Konsolenanwendung, die persönliche Ausgaben erfasst und summiert. Startet in einem komplett leeren Ordner.

WARUM es das gibt

Claude Code ist kein Chat. Ein Chat gibt dir Text, den du dann von Hand in Dateien kopierst. Claude Code ist ein **Agent**: Er liest deine Dateien selbst, ändert sie selbst, führt selbst Befehle im Terminal aus, sieht den Fehler, den er verursacht hat, und behebt ihn selbst. Deine Arbeit verschiebt sich von "Ich tippe Code" zu "Ich beschreibe, was ich will, und prüfe das Ergebnis". Der ganze Kurs ist im Grunde das Erlernen dieser neuen Fähigkeit: **einem Agenten präzise sagen, was du willst, und prüfen, ob er es wirklich getan hat.**

Theorie des Moduls (einmal lesen, bei Bedarf zurückkommen)

Zwei Arbeitsmodi:

- **Interaktiv:** tippe `claude` im Projektordner → es öffnet sich ein Gespräch, das läuft, bis du es beendest. Hier arbeitest du 95% der Zeit.
- **Einmalig:** `claude -p "Frage oder Aufgabe"` → Claude erledigt es, gibt das Ergebnis aus und beendet sich. Für Skripte und schnelle Checks (mehr in Modul 6).

Grundbefehle im Gespräch:

Befehl	Was er tut	Warum du ihn brauchst
<code>/help</code>	Liste aller Befehle	das Erste, was man in einem neuen Tool tippt
<code>/clear</code>	löscht das ganze Gespräch	neue Aufgabe = sauberer Kontext = bessere Ergebnisse
<code>/compact</code>	komprimiert das Gespräch, um Platz zu schaffen	lange Sitzungen füllen den "Kontext" — Claude hat begrenztes Arbeitsgedächtnis
<code>/model</code>	wechselt das Modell (schneller/günstiger vs. klüger)	für Kleinigkeiten brauchst du nicht das stärkste Modell
<code>/resume</code>	kehrt zu einer früheren Sitzung zurück	mach morgen weiter, wo du aufgehört hast
<code>/rewind</code>	setzt das Gespräch (und auf Wunsch Dateien) auf einen früheren Punkt zurück	"Rückgängig", wenn Claude in die falsche Richtung läuft
<code>Esc</code>	unterbricht Claude mitten in der Arbeit	siehst du, dass er das Falsche tut? Warte nicht bis zum Ende

Dateien referenzieren: tippe `@` gefolgt vom Dateinamen (z.B. `@ausgaben.py`) — Claude liest diese Datei garantiert. Ohne das entscheidet er selbst, was er liest (meist richtig, aber `@` ist die Garantie).

Berechtigungsmodi (Permission Modes) — Claude fragt um Erlaubnis, bevor er etwas ändert oder ausführt. WARUM: Ein Agent, der selbstständig Dateien ändert und Befehle ausführt, braucht eine Bremse. Die Modi (Wechsel mit **Shift+Tab**):

- **default** — fragt bei jeder Änderung/jedem Befehl. Fang hier an: Du siehst genau, was er tut.

- **acceptEdits** — Dateiänderungen laufen automatisch durch, Befehle fragen weiterhin. Wenn Vertrauen da ist.
- **plan** — Claude darf nur LESEN und einen Plan machen, ändert nichts (Modul 3).
- **bypassPermissions** — fragt gar nichts. Nur für isolierte/sichere Umgebungen — vorerst überspringen.

Woche 1 — erste Schritte und die erste Anwendung

Tag 1 — Installation und erstes Gespräch

1. Erstelle einen leeren Ordner `ausgaben-tagebuch` und wechsele im Terminal hinein.
2. Starte `claude`. Tippe `/help` und lies, was es gibt (du musst dir nichts merken).
3. **Ü1.1:** Bitte Claude, eine CSV-Datei mit 10 ausgedachten Ausgaben (Datum, Beschreibung, Kategorie, Betrag) und ein Python-Skript `ausgaben.py` zu erstellen, das die CSV liest und die Gesamtsumme ausgibt. Führe das Skript aus. **Lösung**
4. Achte darauf: Claude hat vor dem Schreiben der Datei und vor dem Ausführen des Befehls um Erlaubnis gefragt. Lies, WAS er genau fragt, bevor du zustimmst — diese Gewohnheit ist Gold wert.

Tag 2 — Ausbau: neue Ausgaben erfassen

1. Mach im selben Ordner weiter (`claude` , und wenn du das alte Gespräch willst: `/resume`).
2. **Ü1.2:** Verlange ein Menü: eine Option zum Erfassen einer neuen Ausgabe (wird an die CSV angehängt) und eine Option zur Anzeige der Summe. Teste beides. **Lösung**
3. Probiere `@ausgaben.py` in einer Nachricht — sieh, wie Claude reagiert, wenn du ihm die Datei explizit gibst.

Tag 3 — Monatsbericht

1. **Ü1.3:** Verlange einen Bericht nach Monaten und Kategorien (z.B. "Essen: 45,00 €, Verkehr: 12,00 €"). Lass Claude das Ausgabeformat selbst vorschlagen und verfeinere es dann: "richte die Spalten aus", "füge unten eine Gesamtsumme hinzu". **Lösung**
2. Der Sinn der Übung: **Du musst das genaue Layout nicht vorher kennen — verfeinere es im Gespräch.** Das ist eine normale Arbeitsweise.

Tag 4 — absichtlich einen Fehler provozieren (Debugging)

1. **Ü1.4:** Öffne die CSV von Hand im Editor und mach eine Zeile kaputt (z.B. lösche den Betrag oder schreib einen Buchstaben statt einer Zahl). Führe das Skript aus — es stürzt ab. Dann tippe in Claude nur: "führe ausgaben.py aus und behebe den Fehler". Beobachte: Er liest den Fehler, findet die Ursache, behebt sie und führt erneut aus, um zu prüfen. **Lösung**
2. WARUM das entscheidend ist: Das ist die Schleife, die du ständig nutzen wirst — **Claude kann seine eigene Arbeit prüfen, wenn du ihm einen Weg gibst, sie zu prüfen** (Ausführen, ein Test, ein Beispiel).

Tag 5 — der Einmal-Modus

1. Beende das Gespräch. **Ü1.5:** Führe im Terminal aus: `claude -p "wie viele Zeilen hat ausgaben.csv und was ist die größte Einzelausgabe?"` **Lösung**
2. Probiere noch 2–3 einmalige Fragen zu deinem Code. Bemerke den Unterschied: kein Gespräch, du bekommst eine Antwort und fertig.

Tag 6 — Retrospektive: Schreib auf (in ein Heft oder eine Datei `NOTIZEN.md`): Was hat mich überrascht? Wann lag Claude daneben und warum — war meine Beschreibung unpräzise? Welchen Befehl habe ich noch nicht probiert?

Woche 2 — Sitzungen, Kontext und Berechtigungsmodi

Tag 1 — Sitzungen fortsetzen

1. Schließe das Terminal komplett. Öffne es neu, geh in den Ordner, starte `claude --resume` (oder `claude` und dann `/resume`) und wähle die gestrige Sitzung.
2. **Ü1.6:** Mach GENAU dort weiter, wo du aufgehört hast: Verlange eine Kategorie "Sonstiges" für Ausgaben ohne Kategorie. Bemerke, dass Claude sich an alles aus dieser Sitzung "erinnert". [Lösung](#)

Tag 2 — Kontext und `/compact`

1. Theorie in 3 Sätzen: Claude hat ein begrenztes "Arbeitsgedächtnis" (Kontext). Alles, was ihr besprochen habt, und alle Dateien, die er gelesen hat, füllen es. Wenn es voll ist, werden ältere Teile zusammengefasst — deshalb werden lange Sitzungen "vergesslich".
2. Praxis: Schau auf die Kontextanzeige (unterer Bildschirmrand). Tippe `/compact` und sieh, was passiert. Regel zum Mitnehmen: **neue Aufgabe** → `/clear`. Zieh nicht ein Gespräch über Tage.

Tag 3 — `/rewind` und Kurskorrektur

1. **Ü1.7:** Verlange absichtlich etwas Vages, z.B. "mach die Ausgabe etwas hübscher". Wenn Claude etwas macht, das dir nicht gefällt, nutze `/rewind`, um zurückzugehen, und wiederhole die Anfrage präzise ("nutze eine Tabelle mit Spalten Datum/Beschreibung/Betrag, Betrag rechtsbündig"). Vergleiche die Ergebnisse. [Lösung](#)
2. Lektion: Qualität des Ergebnisses = Qualität der Beschreibung. `/rewind` + eine bessere Beschreibung ist oft schneller als 5 Korrekturrunden.

Tag 4 — Berechtigungsmodi in der Praxis

1. **Ü1.8:** Wechsle zu **acceptEdits** (Shift+Tab) und verlange eine kleine Verbesserung (z.B. ein schöneres Datum in der Ausgabe). Bemerke, dass Dateiänderungen keine Bestätigung mehr brauchen, Befehle aber weiterhin. Wechsle zurück auf default. [Lösung](#)
2. Denk nach und schreib auf: In welchen Situationen wäre acceptEdits gefährlich? (z.B. ein Ordner mit wichtigen Dateien ohne Git)

Tag 5 — WISSENSCHECK Modul 1 Ü1.9 (ohne Anleitung): Füge in einer neuen Sitzung (`/clear`) mit allem Gelernten den Export des Monatsberichts in eine JSON-Datei hinzu. Bedingungen: (1) du musst mindestens einmal eine `@`-Referenz nutzen, (2) du musst Claude den Code selbst ausführen und prüfen lassen, (3) prüfe am Ende mit einem einmaligen `claude -p`-Befehl, dass die JSON-Datei in Ordnung ist. [Lösung und Erfolgskriterien](#)

Tag 6 — Retrospektive.

Modul 2 — CLAUDE.md und Gedächtnis (Wochen 3–4)

Mini-Projekt: "Persönliche Portfolio-Seite" — eine statische Website (HTML/CSS/JS) mit Startseite, Projektseite und Kontaktformular. Von Null.

WARUM es das gibt

Claude startet jede neue Sitzung **leer** — er weiß nichts über dein Projekt, deine Regeln, deinen Stil.

`CLAUDE.md` ist eine Datei, die Claude zu Beginn JEDER Sitzung in diesem Ordner automatisch liest. Das ist der Unterschied zwischen einem Kollegen, dem du jeden Tag alles neu erklärst — und einem Kollegen, der die "Hausordnung" gelesen hat, bevor er sich an den Tisch setzt.

Theorie des Moduls

Drei Ebenen von CLAUDE.md:

Datei	Gilt für	Beispielinhalt
<code>~/ .claude/CLAUDE.md</code> (global)	ALLE deine Projekte	"antworte mir auf Deutsch", "Commit-Nachrichten kurz halten"
<code>CLAUDE.md</code> im Projektordner	dieses Projekt und alle, die daran arbeiten (kommt ins Git)	Projektregeln, Konventionen, bekannte Fallen
<code>CLAUDE.local.md</code>	dieses Projekt, nur dein Rechner (kommt nicht ins Git)	deine persönlichen Abkürzungen, lokale Pfade

Was in CLAUDE.md GEHÖRT: Regeln, die man dem Code nicht ansieht. Konventionen ("alle Texte der Website sind auf Deutsch"), Verbote ("keine Frameworks, reines HTML/CSS/JS"), Fallen ("Datei X nicht anfassen, sie wird automatisch generiert"), Projektbefehle ("die Website wird durch Öffnen von index.html getestet").

Was NICHT hineingehört: was Claude selbst aus dem Code sehen kann (die Dateiliste, was welche Funktion tut) — das veraltet und füllt den Kontext umsonst.

Die # -Abkürzung: tippe `#` am Anfang einer Nachricht (z.B. `# für Farben immer CSS-Variablen verwenden`) — Claude schreibt es dauerhaft für dich ins Gedächtnis/CLAUDE.md.

`/init`: ein Befehl, der ein bestehendes Projekt analysiert und selbst einen ersten CLAUDE.md-Entwurf erstellt.

Woche 3 — der Schmerz ohne CLAUDE.md, dann das Heilmittel

Tag 1 — eine Website ohne Regeln (absichtlich)

1. Neuer Ordner `portfolio`. **Ü2.1:** Verlange eine Startseite: Name, kurze Vorstellung, 3 Abschnitte (Über mich / Projekte / Kontakt). Gib KEINE Regeln zu Stil/Sprache vor. Nimm einfach an, was er anbietet.

Lösung

Tag 2 — "Vergesslichkeit" in Aktion

1. Schließe die Sitzung. Öffne eine NEUE (ohne `--resume`!). **Ü2.2:** Verlange eine neue Seite `projekte.html` mit 3 Projektkarten. Vergleiche sie mit der Startseite: Farben? Schrift? Sprache der

Texte? Struktur? Sehr wahrscheinlich passt etwas nicht — Claude konnte deine Konventionen nicht kennen. Schreib auf, was konkret abweicht. [Lösung](#)

Tag 3 — wir schreiben CLAUDE.md

1. **Ü2.3:** Erstelle eine `CLAUDE.md` im Projektordner (du kannst sie auch von Claude nach deinem Diktat schreiben lassen) mit mindestens 5 Regeln, z.B.: Sprache aller Texte, Farbpalette (konkrete Hex-Werte), keine Frameworks und Bibliotheken, jede Seite nutzt eine gemeinsame `style.css`, Dateinamen kleingeschrieben. [Lösung mit komplettem Beispiel](#)

Tag 4 — Test des Heilmittels

1. Neue Sitzung (kein resume). **Ü2.4:** Verlange eine dritte Seite (`kontakt.html` mit Formular). Erwähne KEINE einzige Regel. Prüfe: Hat er CLAUDE.md ohne Erinnerung befolgt? Wenn eine Regel nicht befolgt wurde — ist sie vermutlich unklar formuliert; schärfe die Formulierung nach. [Lösung](#)

Tag 5 — die #-Abkürzung und /init

1. **Ü2.5:** Tippe mitten in der Arbeit eine #-Regel, die dir gefehlt hat (z.B. `# Buttons haben immer einen Hover-Effekt`). Schau, wohin er sie geschrieben hat. Führe dann in einem ANDEREN bestehenden Ordner (irgendein altes Projekt oder ein heruntergeladenes Repo) `/init` aus und lies, was er generiert hat — was würdest du streichen/ergänzen? [Lösung](#)

Tag 6 — Retrospektive.

Woche 4 — Hierarchie und Pflege des Gedächtnisses

Tag 1 — die globale CLAUDE.md

1. **Ü2.6:** Erstelle/erweitere die globale `~/.claude/CLAUDE.md` (unter Windows: `C:\Users\<du>\.claude\CLAUDE.md`) mit 2–3 PERSÖNLICHEN Regeln, die du überall willst (Kommunikationssprache, Stil der Commit-Nachrichten...). Öffne eine Sitzung im Portfolio-Projekt und prüfe, dass BEIDE Dateien gelten — die globale und die des Projekts. [Lösung](#)

Tag 2 — was nicht in CLAUDE.md gehört

1. Eine Denkübung: Füge CLAUDE.md einen absichtlich schlechten Eintrag hinzu — eine detaillierte Beschreibung, was jede Datei tut. Bemerke in den nächsten Sitzungen, dass das (a) veraltet, sobald sich der Code ändert, (b) Kontext verschwendet. Lösche es. Regel: **CLAUDE.md enthält, was man dem Code NICHT ansieht.**

Tag 3–4 — die Website ausbauen mit lebendiger CLAUDE.md

1. Füge der Website hinzu, was du willst (Galerie, dunkles Design, Animationen). Jedes Mal, wenn du Claude an etwas ERINNERN musst — ist das ein Kandidat für CLAUDE.md. Ergänze sie nebenbei (die #-Abkürzung).

Tag 5 — WISSENSCHECK Modul 2 Ü2.7 (ohne Anleitung): Simuliere ein "neues Teammitglied": Vergiss alle Sitzungen — öffne eine komplett neue Sitzung und verlange: "füge eine Seite mit einer Preisliste der Leistungen hinzu". Erfolgskriterium: Die neue Seite passt zur Website (Sprache, Farben, Struktur, Stil) OHNE eine einzige Erinnerung von dir. Wenn nicht — hat CLAUDE.md ein Loch; finde und stopfe es. [Lösung und Kriterien](#)

Tag 6 — Retrospektive.

Modul 3 — Plan-Modus und Loop (Wochen 5–6)

Mini-Projekt: "Quiz-App" — ein Web-Quiz: Fragen mit 4 Antworten, Zeitmessung, Endergebnis und Bestenliste (localStorage). Von Null.

WARUM es das gibt

Bei kleinen Änderungen kann Claude sofort loslegen — irrt er sich, ist die Korrektur billig. Bei GRÖßEREN Vorhaben (neue App, Refactoring, eine Änderung über 10 Dateien) bedeutet eine falsche Richtung eine Stunde Aufräumen. Der **Plan-Modus** dreht die Reihenfolge um: Claude darf zuerst nur LESEN und erkunden, schlägt dir dann einen Plan vor, und erst wenn du den Plan freigibst — darf er ändern. Einen 20-Zeilen-Plan zu prüfen ist unvergleichlich billiger, als 500 Zeilen falschen Code zu prüfen.

Theorie des Moduls

- In den Plan-Modus kommst du mit **Shift+Tab** (bis "plan mode" erscheint) oder indem du `claude` startest und vor der ersten Anfrage umschaltest.
- Im Plan erkundet Claude den Code, stellt Fragen und schlägt am Ende einen Plan vor. Du kannst: **freigeben, Änderungen verlangen** (schreib einfach, was er ändern soll — der Plan wird überarbeitet) oder **abbrechen**.
- Wann der Plan-Modus NICHT nötig ist: Kleinkram ("ändere die Farbe des Buttons"). Übertriebene Zeremonie bei kleinen Dingen bremst.
- **Extended Thinking:** füge bei schweren Problemen "think hard" / "denk gründlich nach" zur Anfrage hinzu — Claude bekommt mehr "Denkzeit" vor der Antwort. Nutze es für: Architekturentwürfe, seltsame Bugs, Entscheidungen mit mehreren Optionen.
- `/loop` : wiederholt eine Aufgabe in Intervallen (z.B. alle 5 Minuten etwas prüfen). Für Überwachung und wiederkehrende Arbeiten.

Woche 5 — das Quiz durch den Plan-Modus

Tag 1 — ein Plan für die Struktur

1. Neuer Ordner `quiz` . Geh in den Plan-Modus. **Ü3.1:** Verlange: "baue eine Web-Quiz-App: Fragen mit 4 Antworten, 10 Allgemeinwissensfragen, Ergebnis am Ende". LIES den Plan sorgfältig vor der Freigabe: Erwähnt er die Dateien, die du erwartest? Ist etwas überkompliziert? [Lösung](#)
2. Gib den Plan frei und lass ihn implementieren. Öffne das Quiz im Browser.

Tag 2 — einen Plan absichtlich ablehnen

1. Plan-Modus. **Ü3.2:** Verlange eine Bestenliste (Spielername + Punkte + Datum, Speicherung in localStorage, Top 10 anzeigen). Wenn der Plan kommt — gib NICHT sofort frei: Verlange mindestens 2 Änderungen (z.B. "Rekorde auch pro Kategorie speichern" oder "einen Knopf zum Löschen der Rekorde hinzufügen"). Sieh zu, wie der Plan überarbeitet wird, und gib erst dann frei. [Lösung](#)
2. Lektion: Der Plan ist ein VORSCHLAG, keine Pflicht. Der größte Wert des Plan-Modus ist das Verhandeln VOR dem Code.

Tag 3 — Zeitmessung + think hard

1. **Ü3.3:** Verlange einen Timer pro Frage (z.B. 15 Sekunden, Zeit abgelaufen = falsche Antwort) und Bonuspunkte für Schnelligkeit. Füge zur Anfrage hinzu: "denk gründlich über die Randfälle nach: was, wenn der Nutzer den Tab wechselt, was, wenn er in der letzten Sekunde antwortet". Vergleiche die Qualität des Plans mit einem gewöhnlichen. [Lösung](#)

Tag 4 — wann der Plan-Modus NICHT nötig ist

1. Ohne Plan-Modus, direkt: 3–4 kleine Änderungen ("größere Schrift der Frage", "grün/rot färben der Antworten", "füge 5 weitere Fragen hinzu"). Spüre den Unterschied im Tempo. Notiere deine Faustregel: Ab welcher Aufgabengröße lohnt sich der Plan-Modus FÜR DICH?

Tag 5 — /loop

1. **Ü3.4:** Probiere `/loop` an einer harmlosen Aufgabe, z.B.: "prüfe alle 5 Minuten, ob es im Ordner neue .html-Dateien ohne entsprechenden Link in index.html gibt, und sag mir Bescheid". Lass es 2–3 Runden laufen und stoppe dann. Überlege: Was wäre in DEINER echten Arbeit so eine Überwachung wert? [Lösung](#)

Tag 6 — Retrospektive.

Woche 6 — ein großes Refactoring per Plan

Tag 1–2 — Vorbereitung und der Refactoring-Plan

1. Dein Quiz steckt vermutlich in 1–2 Dateien. **Ü3.5:** Verlange im Plan-Modus ein Refactoring in Module: getrennte Dateien für Fragen (Daten), Quiz-Logik, Timer, Rekorde und Darstellung. Bedingung in der Anfrage: "nach dem Refactoring muss alles identisch funktionieren wie vorher". Prüfe den Plan IM DETAIL — beim Refactoring geht am leichtesten nebenbei etwas kaputt. [Lösung](#)
2. Gib frei und lass laufen. Teste danach ALLE Quiz-Funktionen (spiel eine ganze Runde, prüfe die Rekorde).

Tag 3–4 — Ausbau auf der neuen Struktur

1. Füge Fragenkategorien und eine Kategorieauswahl am Anfang hinzu. Bemerke: Ist die Änderung auf der modularen Struktur LEICHTER? Das war der Sinn des Refactorings.

Tag 5 — WISSENSCHECK Modul 3 Ü3.6 (ohne Anleitung): Durch den Plan-Modus, vom Plan bis zur Implementierung: Füge dem Quiz einen Modus "2 Spieler abwechselnd" mit einem Vergleichsergebnis am Ende hinzu. Kriterien: Du hast den Plan vor der Freigabe mindestens einmal überarbeiten lassen; nach der Implementierung hast du persönlich beide Modi getestet (1 und 2 Spieler). [Lösung und Kriterien](#)

Tag 6 — Retrospektive.

Modul 4 — MCP-Server (Wochen 7–8)

Mini-Projekt: "Wetter + Notizen" — eine App, die die Wettervorhersage anzeigt (externe Daten) und deine Tagesnotizen (lokale SQLite-Datenbank).

WARUM es das gibt

Von Haus aus "sieht" Claude nur zwei Dinge: deine Dateien und dein Terminal. Er sieht nicht deine Datenbank, deinen Browser, dein Slack, dein GitHub-Konto. **MCP (Model Context Protocol)** ist der Standardweg, ihm neue "Sinne" zu geben — ein MCP-Server ist ein kleines Programm, das Claude Werkzeuge bereitstellt ("lies die Tabelle", "öffne eine Seite im Browser", "sende eine Nachricht"). Statt dass du Claude nacherzählst, was in der Datenbank steht, schaut er selbst nach.

Theorie des Moduls

Befehle:

- `claude mcp add <name> ...` — fügt einen Server hinzu (die genaue Syntax hängt vom Server ab; jeder Server hat in seiner Doku einen fertigen Befehl zum Kopieren)
- `claude mcp list` — Liste der verbundenen Server und ihr Status
- `claude mcp remove <name>` — Entfernen
- `/mcp` in der Sitzung — Serverstatus und Anmeldung (für Server, die ein Konto brauchen)

Scope (Geltungsbereich) — wo die Konfiguration gespeichert wird:

Scope	Gilt für	Wann
<code>local</code> (Standard)	nur du, nur dieses Projekt	Ausprobieren, persönliche Werkzeuge
<code>project</code>	alle im Projekt (die Datei <code>.mcp.json</code> kommt ins Git)	das Team teilt dieselben Werkzeuge
<code>user</code>	du, in allen Projekten	Werkzeuge, die du überall nutzt

Sicherheitshinweis (wichtig!): Ein MCP-Server hat Zugriff auf das, was du ihm gibst. Füge nur Server hinzu, denen du vertraust (offizielle, bekannte). Ein Server, der die Datenbank liest — überlege, ob er auch Schreibzugriff braucht.

Woche 7 — der erste MCP: eine Datenbank

Tag 1 — die App (erst mal ohne MCP)

1. Neuer Ordner `wetter-notizen`. **Ü4.1:** Verlange eine App (Python plus etwas Web ist gut, lass Claude vorschlagen): eine SQLite-Datenbank `notizen.db` mit einer Notiztabelle (Datum, Text), Erfassen und Ansehen der Notizen pro Tag. Füge 5–6 Probenotizen ein. [Lösung](#)

Tag 2 — ein SQLite-MCP-Server

1. **Ü4.2:** Finde und füge einen MCP-Server für SQLite hinzu (frag Claude selbst: "was ist der empfohlene Weg, einen MCP-Server für sqlite hinzuzufügen, und gib mir den genauen Befehl für meine Datenbank `notizen.db`"). Danach: `claude mcp list` muss ihn anzeigen. [Lösung](#)

2. Testfrage an Claude: "wie viele Notizen sind in der Datenbank und welche ist die neueste?" — er muss durch Lesen ÜBER MCP antworten, nicht durch das Schreiben eines Skripts.

Tag 3 — den Unterschied spüren

1. **Ü4.3:** Mach dieselbe Aufgabe auf beide Arten und vergleiche: (a) frag Claude etwas über die Daten, WÄHREND der MCP aktiv ist; (b) entferne den Server (`claude mcp remove`) und stell dieselbe Frage — jetzt muss er ein Skript/einen Befehl schreiben, um an die Daten zu kommen. Notiere den Unterschied in Geschwindigkeit und Schrittzahl. Füge den Server wieder hinzu. [Lösung](#)

Tag 4 — externe Daten (das Wetter)

1. **Ü4.4:** Füge der App eine Wettervorhersage für deine Stadt hinzu (es gibt kostenlose APIs ohne Registrierung — lass Claude eine vorschlagen und den Aufruf AUS DEM CODE der App implementieren). [Lösung](#)
2. Diskussion mit dir selbst (schreib sie auf): Das Wetter holt der CODE der App per API, die Datenbank liest CLAUDE per MCP. Worin liegt der Unterschied? (Antwort in Lösung Ü4.7.)

Tag 5 — Retrospektive + Aufräumen.

Woche 8 — der Browser-MCP und die Registry

Tag 1–2 — der Browser-MCP

1. **Ü4.5:** Verbinde einen Browser-MCP (z.B. die "Claude in Chrome"-Erweiterung oder einen Playwright-MCP — frag Claude, was für deine Umgebung verfügbar ist und wie man es installiert). Aufgabe: Lass Claude SELBST deine App im Browser öffnen, über das Formular eine neue Notiz eintragen und prüfen, dass sie in der Liste erscheint. [Lösung](#)
2. WARUM das groß ist: Bis jetzt warst DU Claudes Augen für alles im Browser. Jetzt kann er selbst klicken und die Seite lesen — er kann selbst testen, was er baut.

Tag 3 — die MCP-Server-Registry

1. **Ü4.6:** Erkunde, was es gibt: Frag Claude nach bekannten/offiziellen MCP-Servern oder suche im Browser nach "MCP servers registry". Wähle EINEN, der dir persönlich nützlich erscheint (GitHub? Dateisystem? etwas Drittes?), füge ihn hinzu und denk dir eine Mini-Aufgabe aus, um ihn auszuprobieren. [Lösung](#)

Tag 4 — .mcp.json zum Teilen

1. Füge einen deiner Server mit `--scope project` hinzu und schau dir die entstandene `.mcp.json` im Projekt an. Das ist die Datei, die ein Kollege über Git bekäme und damit dieselben Werkzeuge hätte wie du.

Tag 5 — WISSENSCHECK Modul 4 Ü4.7 (ohne Anleitung, Theorie + Praxis): (1) Erkläre in eigenen Worten, in 5 Sätzen, den Unterschied zwischen einem MCP-Server und einem API-Aufruf aus dem App-Code — und nenne je ein Beispiel, wann was die richtige Wahl ist. (2) Praktisch: Lass Claude mit Browser-MCP + SQLite-MCP zusammen die ganze App testen und dir einen kurzen Bericht schreiben, was funktioniert und was nicht. [Lösung und Kriterien](#)

Tag 6 — Retrospektive.

Modul 5 — Parallele Subagenten (Wochen 9–10)

Mini-Projekt: Analyse eines fremden Open-Source-Projekts + ein eigener "Rezensent"-Agent, der dein Quiz aus Modul 3 begutachtet.

WARUM es das gibt

Zwei Probleme, die Subagenten lösen:

1. **Der Kontext ist begrenzt.** Wenn Claude 50 Dateien lesen muss, um eine Frage zu beantworten, füllen diese 50 Dateien für immer deine Sitzung. Ein Subagent ist ein "Kundschafter": Er zieht los, liest alles in SEINEM eigenen Kontext und bringt dir nur ein 10-Zeilen-Fazit zurück.
2. **Manche Arbeiten sind unabhängig.** Drei Fragen zu drei verschiedenen Codeteilen müssen nicht aufeinander warten — drei Subagenten arbeiten parallel.

Wann sie sich NICHT lohnen: kleine Aufgaben. Ein Subagent startet "bei Null" (er sieht euer Gespräch nicht), bei einer Kleinigkeit verbraucht er also mehr fürs Einarbeiten als für die Arbeit.

Theorie des Moduls

- Subagenten startest du in natürlicher Sprache: "schick einen Subagenten, um X zu untersuchen", "starte 3 Agenten parallel: der erste soll..., der zweite..., der dritte...".
- Eingebaute Typen: **Explore** (nur lesen — ein sicherer Kundschafter), **Plan** (entwirft ein Vorgehen), **general-purpose** (kann alles).
- **Eigene Agenten** — deine Spezialisten: eine Datei in `.claude/agents/` mit Rollenbeschreibung, Anweisungen und (optional) eingeschränkten Werkzeugen. Sie lassen sich auch mit dem Befehl `/agents` anlegen. WARUM: Statt jedes Mal denselben langen Prompt zu tippen ("prüfe den Code auf diese 10 Dinge"), schreibst du ihn einmal als Agenten auf.

Woche 9 — Kundschafter auf fremdem Code

Tag 1 — das Ziel klonen

1. Wähle ein bekanntes Open-Source-Projekt mittlerer Größe und klonе es, z.B.: `git clone https://github.com/pallets/flask` (oder ein beliebiges Projekt, das dich interessiert). Geh in den Ordner, starte `claude`.

Tag 2 — ein Kundschafter

1. **Ü5.1:** Schick einen Explore-Agenten: "schick einen Subagenten, der untersucht, wie in diesem Projekt die Fehlerbehandlung organisiert ist, und mir einen kurzen Bericht mit den Pfaden der Schlüsseldateien zurückbringt". Bemerge: Deine Sitzung blieb "leicht" — der Agent hat in seinem eigenen Kontext gelesen. **Lösung**

Tag 3 — drei parallele Kundschafter

1. **Ü5.2:** Eine Anfrage, drei Agenten parallel, drei verschiedene Fragen zum Projekt (z.B.: wie Tests geschrieben werden / wie die Konfiguration organisiert ist / welche externen Bibliotheken es nutzt und wofür). Bitte Claude, die Ergebnisse am Ende zu einer Übersicht zusammenzuführen. **Lösung**

Tag 4 — Vergleich mit/ohne

1. **Ü5.3:** Stell dieselbe Art Frage (eine neue Frage zum Projekt) einmal DIREKT (ohne Subagent, er soll die Dateien in deiner Sitzung lesen) und einmal ÜBER einen Subagenten. Vergleiche: Kontextverbrauch (die Anzeige), Zeit, Antwortqualität. Notiere, wann du welchen Ansatz wählen würdest. [Lösung](#)

Tag 5 — Retrospektive.

Woche 10 — dein eigener Spezialist

Tag 1–2 — der eigene "Rezensent"-Agent

1. **Ü5.4:** Erstelle einen eigenen Agenten in `.claude/agents/rezensent.md` (über `/agents` oder lass ihn von Claude erstellen). Rolle: prüft Code anhand einer Checkliste — Lesbarkeit, Variablennamen, Code-Wiederholung, offensichtliche Bugs, Kommentare nur wo nötig. Er soll READ-ONLY sein (nur lesen, nichts ändern) und den Bericht als Liste von Befunden mit Pfad und Vorschlag zurückgeben. [Lösung mit komplettem Beispiel](#)

Tag 3 — der Rezensent bei der Arbeit

1. **Ü5.5:** Geh zurück in den Quiz-Ordner aus Modul 3 (kopiere den Agenten hinüber oder lege ihn auf User-Ebene an). Lass den Rezensenten über das ganze Quiz laufen. Lies die Befunde KRITISCH: Womit stimmst du überein, womit nicht? Behebe 2–3 Befunde (die Korrekturen kannst du auch vom normalen Claude verlangen). [Lösung](#)

Tag 4 — den Agenten nachjustieren

1. Der Rezensent hat bestimmt auch Unwichtiges gemeldet. Verfeinere seine Anweisungen (z.B. "melde keine Stil-Kleinigkeiten, wenn sie nicht in CLAUDE.md vereinbart sind") und lass ihn erneut laufen. WARUM: Ein Agent wird durch Iterationen kalibriert, wie ein Kollege.

Tag 5 — WISSENSCHECK Modul 5 Ü5.6 (ohne Anleitung): An einem Open-Source-Projekt deiner Wahl:

(1) drei parallele Explore-Agenten mit drei Fragen, die DU dir ausdenkst, (2) ein zusammengeführter Bericht, (3) deine Einschätzung in 5 Sätzen: Was wäre in diesem Projekt am schwersten zu ändern und warum — auf Basis des Berichts. Kriterium: Der Bericht enthält konkrete Dateipfade, und deine Einschätzung bezieht sich auf die Befunde. [Lösung und Kriterien](#)

Tag 6 — Retrospektive.

Modul 6 — Git-Integration und Automatisierung (Wochen 11–12)

Abschlussprojekt: eine komplette kleine Anwendung nach DEINER Wahl — von der Idee bis zum GitHub-Repository, unter Anwendung von allem Gelernten.

WARUM es das gibt

Bisher hast du Code produziert. Dieses Modul handelt davon, wie Code in echter Arbeit LEBT: Versionen (Git), Prüfung der Änderungen vor dem Zusammenführen (Review — ein Bug im Review ist billiger als in Produktion) und Automatisierung (Hooks, eigene Befehle, Headless-Modus) — damit sich der wiederkehrende Teil deiner Arbeit verpacken und von selbst ausführen lässt.

Theorie des Moduls

- **Git durch Claude:** Claude beherrscht den ganzen Git-Ablauf — `git init`, Branches, sinnvolle Commit-Nachrichten (er sichtet die Änderungen selbst und verfasst die Nachricht), und mit dem **GitHub CLI** (`gh`) auch PRs. Du GENEHMIGST weiterhin jeden Befehl.
- `/code-review` — prüft deine aktuellen Änderungen und meldet Bugs/Probleme, bevor du sie committest.
- **Hooks** — automatische Aktionen bei Ereignissen (z.B. "nach jeder Dateiänderung die Tests laufen lassen"). Definiert in `.claude/settings.json`. Das wichtige WARUM: Einen Hook führt das Tool selbst aus (garantiert, jedes Mal), nicht Claude nach eigenem Ermessen — Regeln, die passieren MÜSSEN, gehören in einen Hook, nicht in CLAUDE.md.
- **Eigene Slash-Befehle** — dein Workflow als Befehl: eine Datei in `.claude/commands/` (z.B. `liefern.md`) wird zu `/liefern`. Statt jedes Mal dieselbe Abfolge von Anfragen zu tippen.
- **Headless-Modus** — `claude -p "Aufgabe"` aus einem Skript/CI: Claude als Werkzeug in der Automatisierung, ohne Gespräch.

Woche 11 — ein Git-Ablauf von Null

Tag 1 — das Repo und der erste Commit

1. Wähle eine Idee für das Abschlussprojekt (Vorschläge: eine geteilte Einkaufsliste, ein Trainingstagebuch, ein RSS-Nachrichtenleser, ein Aufgabenplaner — oder etwas Eigenes). Neuer Ordner.
2. **Ü6.1:** Gleich am Anfang: Verlange von Claude `git init`, eine `.gitignore` passend zum Projekttyp und eine CLAUDE.md (Modul-2-Wissen!). Dann das erste Feature und der erste Commit — aber LIES die vorgeschlagene Commit-Nachricht, bevor du zustimmst. **Lösung**

Tag 2 — Branch + Feature + Review

1. **Ü6.2:** Ein neues Feature komplett auf einem Branch: "erstelle einen Branch für Feature X, implementiere es" → dann VOR dem Commit `/code-review` ausführen. Lies die Befunde, behebe die berechtigten, dann committe. **Lösung**

Tag 3 — GitHub und ein PR

1. Lege ein (kostenloses) GitHub-Konto an, falls du keins hast, und installiere das `gh` CLI (`gh auth login`).

2. **Ü6.3:** Bitte Claude, das Repo auf GitHub zu veröffentlichen und einen Pull Request von deinem Branch zu öffnen, mit einer Beschreibung der Änderungen. Schau dir den PR im Browser an — das ist das Format, in dem Teams Code austauschen. Führe ihn zusammen (Merge). [Lösung](#)

Tag 4 — ein Hook

1. **Ü6.4:** Richte einen Hook ein, der nach jeder Dateiänderung durch Claude läuft — für den Anfang etwas Einfaches und Sichtbares (z.B. bei einem Python-Projekt: eine Syntaxprüfung der geänderten Datei). Teste ihn: Verlange eine Änderung und vergewissere dich, dass der Hook ausgelöst hat. [Lösung mit settings.json-Beispiel](#)
2. Überlege: Welche Regel aus deiner CLAUDE.md sollte eigentlich ein Hook sein? (Merksatz: CLAUDE.md = Bitte, Hook = Garantie.)

Tag 5 — Retrospektive.

Woche 12 — eigener Befehl, Headless und die ABSCHLUSSPRÜFUNG

Tag 1 — ein eigener Slash-Befehl

1. **Ü6.5:** Erstelle `.claude/commands/liefern.md` — einen `/liefern`-Befehl, der: die Code-Prüfungen ausführt, aus den Änderungen eine Commit-Nachricht verfasst, committet und pusht. Probiere ihn an einer echten Änderung. [Lösung mit komplettem Beispiel](#)

Tag 2 — Headless

1. **Ü6.6:** Schreibe (mit Claudes Hilfe) ein Skript, das `claude -p` nutzt, um täglich eine Frage zum Projekt beantworten zu können, z.B.: "liste die TODO-Kommentare im Code auf und schlage die Reihenfolge der Abarbeitung vor". Führe es von Hand aus und schau dir die Ausgabe an. [Lösung](#)

Tag 3–5 — ABSCHLUSSPRÜFUNG Ü6.7: Stelle dein Projekt fertig und wende ALLES an — Checkliste:

- ☐ CLAUDE.md existiert von Anfang an und wird gepflegt (Modul 2)
- ☐ Mindestens ein größeres Feature durch den Plan-Modus umgesetzt (Modul 3)
- ☐ Mindestens ein MCP-Server im Einsatz (Modul 4)
- ☐ Mindestens eine Untersuchung durch einen Subagenten (Modul 5)
- ☐ Der volle Git-Ablauf: Branches, Review vor dem Commit, ein PR auf GitHub (Modul 6)
- ☐ Mindestens ein Hook oder eigener Befehl im Projekt (Modul 6)
- ☐ Die Anwendung FUNKTIONIERT — du hast sie persönlich von Anfang bis Ende getestet

Erfolgskriterien

Tag 6 — Abschluss-Retrospektive: Was vom Gelernten bleibt in deiner täglichen Arbeit? Schreibe deine eigenen "Top-5-Regeln für die Arbeit mit Claude Code".

Nach dem Kurs

- **Offizielle Dokumentation:** <https://code.claude.com/docs> — besonders die Teile zu Hooks, Skills und dem SDK
 - **Anthropics Best-Practices-Artikel** zu agentic coding (suche nach "Claude Code best practices")
 - Im Tool selbst: `/help` und Fragen wie "wie mache ich X in Claude Code" — das Tool kann sich selbst erklären
 - Ideen zum Weitermachen: eigene Skills (`.claude/skills/`), das Claude Agent SDK (eigene Agenten bauen), fortgeschrittenere MCP-Integrationen
-
-

LÖSUNGEN

Regel: Schau eine Lösung erst nach mindestens 15 Minuten eigenen Versuchs an. Die Prompts unten sind REFERENZEN — deine Formulierung muss nicht gleich sein, sie muss nur dieselben Schlüsselinformationen enthalten. Wenn dein Ergebnis funktioniert, aber von der Lösung abweicht — ist das ein Erfolg, kein Fehler.

Modul 1 — Lösungen

Lösung Ü1.1

Referenz-Prompt:

Erstelle eine CSV-Datei `ausgaben.csv` mit den Spalten `datum`, `beschreibung`, `kategorie`, `betrag` und 10 ausgedachten Zeilen (Daten aus den letzten 2 Monaten, Kategorien: `essen`, `verkehr`, `rechnungen`, `freizeit`). Erstelle dann ein Python-Skript `ausgaben.py`, das die CSV liest und die Summe aller Beträge ausgibt. Führe es aus, um zu prüfen, dass es funktioniert.

Erwartetes Ergebnis: zwei Dateien im Ordner; Claude hat das Skript selbst ausgeführt und dir die Summe gezeigt. **Häufige Fehler:**

- *Ein Prompt ohne Spaltenstruktur* ("mach mir irgendeine CSV mit Ausgaben") — du bekommst zufällige Spalten, die du später umbauen musst. Je früher du die Datenstruktur festlegst, desto weniger Umbauten später.
- *Nicht verlangt, dass er das Skript ausführt* — Claude kann sonst ungeprüften Code abliefern. Gewohnheit: "...und führe es zur Prüfung aus" am Ende der Anfrage.

Lösung Ü1.2

Referenz-Prompt:

Füge `ausgaben.py` ein Menü hinzu mit den Optionen: 1) neue Ausgabe erfassen (fragt nach Datum, Beschreibung, Kategorie, Betrag und hängt eine Zeile an die CSV an), 2) Gesamtsumme anzeigen, 3) beenden. Führe es nach der Änderung aus und teste das Erfassen einer Ausgabe.

Erwartetes Ergebnis: das Menü funktioniert; die neue Ausgabe steht in der CSV-Datei. **Häufige Fehler:** die drei Optionen in drei getrennten Nachrichten zu verlangen — geht, ist aber langsamer; verlange Zusammengehöriges zusammen. Das andere Extrem — 10 Funktionen in einer Nachricht — schadet auch (schwerer zu prüfen). Goldene Mitte: eine runde Einheit pro Anfrage.

Lösung Ü1.3

Referenz-Prompt (erster Schritt):

Füge eine Option 'Monatsbericht' hinzu: Gib für einen gewählten Monat die Summe pro Kategorie und die Gesamtsumme aus. Schlag das Ausgabeformat selbst vor.

dann Verfeinern im Gespräch: `Richte die Beträge rechtsbündig aus und füge eine Linie über der Gesamtsumme hinzu.` **Erwartetes Ergebnis:** ein lesbarer Bericht; du hast mindestens eine

Verfeinerungsrunde gemacht. **Der Punkt:** "schlag selbst vor" ist eine legitime Technik — Claude schlägt oft Besseres vor, als du spezifiziert hättest, und du korrigierst dann nur.

Lösung Ü1.4

Referenz-Prompt: Führe `ausgaben.py` aus – es stürzt ab. Finde die Ursache und behebe sie, dann führe erneut aus zur Bestätigung. **Erwartetes Ergebnis:** Claude führt aus, liest den Traceback, findet die kaputte Zeile (oder schreibt robusteren Code, der schlechte Zeilen überspringt/meldet) und BEWEIST die Korrektur durch erneutes Ausführen. **Häufige Fehler:**

- Den Fehler in eigenen Worten nacherzählen, statt ihn ihn selbst sehen zu lassen — die exakte Information geht verloren. Lass ihn AUSFÜHREN und das Original sehen.
- Die Korrektur ohne erneutes Ausführen akzeptieren — verlange immer den Beweis.

Bonusfrage zum Nachdenken: Ist es besser, dass das Skript bei einer schlechten Zeile abstürzt oder sie still überspringt? (Antwort: kommt darauf an — aber "stilles" Überspringen ohne jede Meldung ist die schlechteste Option; verlange mindestens eine Warnung.)

Lösung Ü1.5

Befehl: `claude -p "wie viele Zeilen hat ausgaben.csv und was ist die größte Einzelausgabe?"`

Erwartetes Ergebnis: eine Antwort im Terminal, ohne ein Gespräch zu betreten. **Wann nutzen:** schnelle Checks, Skripte, Automatisierung. Für alles mit Anschlussfragen — ist der interaktive Modus besser.

Lösung Ü1.6

Vorgehen: `claude --resume` → gestrige Sitzung wählen → Füge eine Kategorie 'Sonstiges' hinzu: Jede ohne Kategorie erfasste Ausgabe bekommt sie automatisch. **Erwartetes Ergebnis:** Claude macht mit vollem Kontextwissen weiter — du musst das Projekt nicht erklären. **Häufiger Fehler:** eine normale neue Sitzung (`claude`) starten und sich wundern, dass er "nicht weiß", wovon du redest. Neue Sitzung = leeres Gedächtnis (außer CLAUDE.md — Modul 2!).

Lösung Ü1.7

Vorgehen: nach der vagen Anfrage und dem schlechten Ergebnis → `/rewind` → Punkt vor der Änderung wählen → eine präzise Anfrage:

Ordne die Ausgabenanzeige in eine Tabelle mit Spalten: Datum (tt.mm.jjjj), Beschreibung (max. 30 Zeichen), Kategorie, Betrag rechtsbündig mit 2 Dezimalstellen. Trenne die Kopfzeile mit einer Linie ab.

Der Punkt: Vergleiche, was du von "mach die Ausgabe etwas hübscher" und von der präzisen Beschreibung bekommen hast. Der Unterschied, den du siehst = der Wert präzisen Beschreibens.

Lösung Ü1.8

Vorgehen: Shift+Tab bis `acceptEdits` erscheint → eine kleine Änderung verlangen → bemerken, dass die Dateiänderung ohne Frage durchgeht, das Ausführen des Skripts aber weiter fragt. **Antwort auf die Gefahrenfrage:** ein Ordner ohne Git (kein Weg zurück), Dateien, die nicht nur Code sind (Dokumente,

Konfigurationen mit Passwörtern), Arbeit an fremdem/produktivem Code. Regel: acceptEdits erst, wenn es ein Sicherheitsnetz gibt (Git) und Vertrauen in die Aufgabe.

Lösung Ü1.9

Erfolgskriterien (prüfe selbst):

- ☐ Es gibt eine neue Menüoption, die eine JSON-Datei des Monatsberichts erzeugt
- ☐ Die JSON enthält: Monat, Summe pro Kategorie, Gesamtsumme
- ☐ Du hast mindestens einmal `@ausgaben.py` in einer Anfrage genutzt
- ☐ Claude hat den Code mindestens einmal selbst ausgeführt und geprüft
- ☐ Du hast das Ergebnis geprüft mit `claude -p "lies bericht-<monat>.json und sag mir, ob es gültiges JSON ist und ob die Kategoriesummen zur Gesamtsumme passen"`

Referenz-Prompt für den Hauptteil:

```
@ausgaben.py Füge eine Option 'Export als JSON' hinzu: Erzeuge für  
einen gewählten Monat eine Datei bericht-YYYY-MM.json mit der Struktur  
{monat, pro_kategorie: {...}, gesamt}. Führe aus, exportiere einen  
Monat und zeig mir den Dateiinhalt.
```

Häufiger Fehler: alles im alten, langen Gespräch zu machen — der Check verlangte eine NEUE Sitzung (`/clear`), gerade damit du siehst, wie viel vom Kontext abhängt.

Modul 2 — Lösungen

Lösung Ü2.1

Referenz-Prompt:

Erstelle die Startseite einer persönlichen Portfolio-Website: index.html mit den Abschnitten 'Über mich', 'Projekte', 'Kontakt' und ausgedachtem Inhalt. Reines HTML und CSS.

Erwartetes Ergebnis: irgendeine ordentliche Website. Hier kann man nichts falsch machen — der Sinn der Übung liegt in dem, was morgen folgt.

Lösung Ü2.2

Erwartetes Ergebnis: eine neue Seite, die sich von der Startseite wahrscheinlich UNTERSCHIEDET — anderer Farbton, vielleicht andere Sprache der Texte, vielleicht Inline-Styles statt des gemeinsamen CSS. Du hast mindestens 2 konkrete Unterschiede notiert. **Falls sie zufällig perfekt passt:** Schau, wie — Claude hat vermutlich index.html gelesen und imitiert. Das ist gut, aber NICHT garantiert; CLAUDE.md ist die Garantie. (Mach Ü2.3 trotzdem.)

Lösung Ü2.3

Komplettes Beispiel einer CLAUDE.md für das Portfolio-Projekt:

```
# Portfolio-Website

## Regeln
- Alle Texte der Website sind auf Deutsch.
- Reines HTML/CSS/JS – keine Frameworks, Bibliotheken oder CDN-Links.
- Alle Seiten nutzen die gemeinsame style.css – keine Inline-Styles und keine <style>-Blöcke im HTML.
- Farbpalette (nur diese): Hintergrund #1a1a2e, Text #eaeaea, Akzent #e94560, Sekundär #16213e. Als CSS-Variablen in :root definiert – neue Farben werden nicht ohne Absprache eingeführt.
- Dateinamen: klein, ohne Leerzeichen (kontakt.html, nicht Kontakt Seite.html).
- Jede Seite hat denselben Header mit Navigation – beim Hinzufügen einer neuen Seite den Link AUF ALLEN Seiten ergänzen.
- Die Website wird durch Öffnen von index.html im Browser getestet (kein Build-Schritt).
```

Warum genau so: Jede Regel ist (1) prüfbar, (2) etwas, das man dem Code NICHT ansieht oder das leicht verletzt wird, (3) kurz. Ein schlechtes Regelbeispiel: "die Website soll schön sein" — nicht prüfbar.

Lösung Ü2.4

Erwartetes Ergebnis: Die Kontaktseite respektiert Sprache, Farben, gemeinsames CSS und Navigation — ohne Erinnerungen. **Wenn etwas nicht respektiert wurde:** Die Regel ist vermutlich schwammig. Beispiel: "nutze unsere Farben" (schwammig) vs. eine Palette mit Hex-Werten (klar). Schreib die Regel um und teste erneut in einer neuen Sitzung — das ist der normale CLAUDE.md-Feinschliff-Zyklus.

Lösung Ü2.5

Vorgehen für #: du tippst z.B. `# Buttons und Links haben immer einen Hover-Effekt` → Claude fragt/zeigt, wohin er schreibt → prüfe, dass die Regel in CLAUDE.md gelandet ist. **Für /init:** Die generierte Datei enthält üblicherweise: Projektbeschreibung, Struktur, Befehle. Kritischer Blick: Ist etwas davon aus dem Code offensichtlich (Kandidat fürs Löschen)? Fehlt eine Falle, die nur du kennst (Kandidat fürs Ergänzen)? `/init` ist ein STARTPUNKT, kein fertiges Produkt.

Lösung Ü2.6

Beispiel einer globalen `~/ .claude/CLAUDE.md`:

```
# Meine globalen Regeln
- Kommuniziere mit mir auf Deutsch.
- Commit-Nachrichten: kurz, mit einem Verb beginnend
  ("Fügt hinzu...", "Behebt...").
- Wenn du eine Lösung vorschlägst, zuerst ein Satz WAS du vorschlägst,
  dann die Details.
```

Prüfung: Frag in einer Sitzung im Portfolio-Ordner "welche Regeln gelten für dich gerade?" — er sollte die globalen und die Projektregeln aufzählen. Regelkonflikt? Die spezifischere (Projekt-)Regel gewinnt in der Regel — am besten aber gar keine Konflikte bauen.

Lösung Ü2.7

Erfolgskriterien:

- ☐ Die Preislistenseite ist in der Sprache aus deinen Regeln
- ☐ Sie nutzt nur Palettenfarben und die gemeinsame style.css
- ☐ Navigation auf allen Seiten aktualisiert
- ☐ Du hast ihn an KEINE einzige Regel erinnert

Wenn es durchgefallen ist: Finde die GENAUE verletzte Regel → lies sie mit den Augen von jemandem, der das Projekt nicht kennt → formuliere sie konkreter → wiederhole den Test in einer neuen Sitzung. Dieser Zyklus (Test in sauberer Sitzung → Loch → Flickern) ist auch die professionelle Praxis für CLAUDE.md.

Modul 3 — Lösungen

Lösung Ü3.1

Referenz-Prompt (im Plan-Modus):

Baue eine Web-Quiz-App: 10 Allgemeinwissensfragen, jede mit 4 Antworten, von denen 1 richtig ist, eine Frage pro Bildschirm, am Ende ein Ergebnis (X/10). Reines HTML/CSS/JS, funktioniert durch Öffnen von index.html.

Worauf du im Plan vor der Freigabe achtest: die Dateiliste (du erwartest index.html, style.css, etwas für Logik und Fragendaten), die Reihenfolge der Schritte, und dass NICHTS Unnötiges dabei ist (Build-Tools, ein Framework — hast du nicht verlangt). Wenn der Plan etwas enthält, das du nicht verstehst — FRAG vor der Freigabe; genau dafür ist der Plan-Modus da.

Lösung Ü3.2

Beispiel für die Überarbeitung des Plans: der Plan kommt → du antwortest z.B.:

Ändere den Plan: 1) Rekorde sollen auch das Spieldatum speichern,
2) füge einen Knopf 'alle Rekorde löschen' mit Bestätigung vor dem Löschen hinzu.

→ ein überarbeiteter Plan kommt → erst dann gibst du frei. **Erwartetes Ergebnis:** Du siehst, wie sich der Plan durch Verhandeln ändert, ohne eine einzige geschriebene Codezeile. **Häufiger Fehler:** den Plan "pro forma" freigeben und Änderungen NACH der Implementierung verlangen — damit hast du die Implementierung der falschen Version bezahlt.

Lösung Ü3.3

Referenz-Prompt:

Denk gründlich nach (think hard) und mach dann einen Plan: ein 15-Sekunden-Timer pro Frage (abgelaufen = falsch), Bonuspunkte für Antwortgeschwindigkeit. Achte auf die Randfälle: Tab-Wechsel mitten in der Frage, Antwort in der letzten Sekunde, was mit dem Timer zwischen den Fragen passiert.

Erwartetes Ergebnis: ein Plan, der die genannten Randfälle EXPLIZIT behandelt (z.B. den Timer bei Fokusverlust pausieren, oder die klare Entscheidung, nicht zu pausieren). Vergleiche: ohne "think hard" und ohne das Aufzählen der Randfälle ist der Plan typischerweise genau an den Rändern flacher. **Der Punkt:** Das Aufzählen der Randfälle in der Anfrage ist DEINE Aufgabe — du weißt, was dich beunruhigt; "think hard" gibt den Raum, es zu durchdenken.

Lösung Ü3.4

Referenz-Vorgehen: `/loop`, dann Beschreibung der Aufgabe und des Intervalls (genaue Syntax: siehe `/help`). Ein harmloses Beispiel: prüfen, ob jede `.html`-Datei einen Link in der Navigation hat. **Erwartetes Ergebnis:** Die Aufgabe hat sich mindestens zweimal wiederholt; du hast sie problemlos gestoppt. **Ideen fürs echte Leben:** beobachten, ob ein Build durchläuft; periodische Prüfung der TODO-Kommentare; prüfen, dass nach einer Änderungsserie alle Tests grün sind.

Lösung Ü3.5

Referenz-Prompt (Plan-Modus):

```
Refaktoriere das Quiz in Module: fragen.js (nur Daten), quiz.js
(Ablauflogik), timer.js, rekorde.js (localStorage), anzeige.js (DOM).
index.html lädt sie. Das Verhalten der App muss IDENTISCH bleiben –
für den Nutzer ändert sich nichts. Mach einen Plan, der auch enthält,
wie wir prüfen, dass nichts kaputtgegangen ist.
```

Worauf du im Plan achtest: Schritte, die Stück für Stück gehen (nicht "alles auf einmal"), und ein PRÜFSCHRITT am Ende. **Erfolgskriterium:** Nach dem Refactoring hast du eine ganze Runde gespielt + die Rekorde funktionieren + der Timer funktioniert. Ein Refactoring ohne Verhaltensprüfung ist nicht fertig.

Lösung Ü3.6

Erfolgskriterien:

- ☐ Der ganze Ablauf durch den Plan-Modus: Anfrage → Plan → MINDESTENS EINE Überarbeitung von dir → Freigabe → Implementierung
- ☐ 2-Spieler-Modus: abwechselnde Fragen oder abwechselnde Runden (deine Wahl — aber bewusst in der Planphase getroffen)
- ☐ Ein Vergleichsergebnis am Ende + der Sieger
- ☐ Persönlich getestet: eine Runde im 1-Spieler-Modus UND eine im 2-Spieler-Modus

Typische Falle: Der 2-Spieler-Modus wirft Fragen auf, die der Plan beantworten muss (dieselben Fragen für beide? was ist mit der Bestenliste?) — wenn der Plan sie nicht erwähnt hat, hättest du das in der Überarbeitung verlangen sollen.

Modul 4 — Lösungen

Lösung Ü4.1

Referenz-Prompt:

```
Baue eine App 'Wetter und Notizen': eine SQLite-Datenbank notizen.db mit einer Tabelle notizen (id, datum, text) und einer einfachen Oberfläche (schlag eine vor – Python mit kleinem Webserver ist gut) zum Erfassen einer Notiz pro Tag und zum Ansehen der Notizen nach Tagen. Füge 6 Probenotizen über die letzte Woche ein. Führe sie aus und zeig mir, wie ich sie öffne.
```

Erwartetes Ergebnis: Erfassen und Ansehen funktionieren; die Datenbank existiert als Datei.

Lösung Ü4.2

Vorgehen: Frag Claude nach dem genauen Befehl für deine Umgebung — MCP-Server werden mit verschiedenen Befehlen installiert und ändern sich mit der Zeit, "frag das Tool" ist also verlässlicher als das Abschreiben aus einem Kurs. Typischerweise bekommst du etwas der Form `claude mcp add <name> -- <Befehl, der den Server mit dem Pfad zur Datenbank startet>`. **Prüfung:** `claude mcp list` zeigt den Server als aktiv; die Frage "wie viele Notizen sind in der Datenbank?" wird über ein MCP-Werkzeug beantwortet (du siehst den Werkzeugaufruf in der Sitzung), nicht durch ein Skript. **Häufiger Fehler:** den Server hinzufügen und in der ALTEN Sitzung fragen — in der Regel braucht es eine neue/neu gestartete Sitzung, damit der Server sichtbar ist.

Lösung Ü4.3

Erwartete Beobachtung: mit MCP — ein Werkzeugaufruf, eine direkte Antwort. Ohne MCP — Claude schreibt und startet ein Skript (mehr Schritte, mehr Freigaben, langsamer), kommt aber AUCH zur Antwort. **Die Erkenntnis zum Mitnehmen:** MCP ist nicht der einzige Weg — es ist der SCHNELLERE und SAUBERERE Kanal für Dinge, die du oft tust. Für eine einmalige Abfrage ist ein Skript völlig okay; für die tägliche Arbeit mit einer Datenbank lohnt sich MCP.

Lösung Ü4.4

Referenz-Prompt:

```
Füge der App eine aktuelle Wetteranzeige für [deine Stadt] hinzu: Nutze eine kostenlose API ohne Registrierung (schlag eine vor und begründe die Wahl), zeige Temperatur und Wetterlage auf der Notizenseite. Handle auch den Fall, dass die API nicht antwortet – die App darf nicht abstürzen.
```

Erwartetes Ergebnis: das Wetter auf der Seite; die App überlebt auch ohne Internet (probier es aus!).

Lösung Ü4.5

Vorgehen: Installation eines Browser-MCP nach der Anleitung für deine Umgebung (frag Claude, was verfügbar ist). Dann:

Öffne die App im Browser, trage über das Formular eine neue Notiz "Test aus dem Browser" für heute ein und prüfe, dass sie in der Liste erscheint. Berichte, was genau du getan und was du gesehen hast.

Erwartetes Ergebnis: Claude klickt/tippt/liest die Seite selbst und liefert einen Bericht. **Kriterium:** Die Notiz existiert wirklich in der Datenbank (prüfe über den SQLite-MCP — eine schöne Gelegenheit, beide zu kombinieren).

Lösung Ü4.6

Es gibt keine einzig richtige Lösung — das Kriterium ist der Prozess: (1) du hast geschaut, was es gibt, (2) einen Server mit BEGRÜNDUNG gewählt, was er dir bringt, (3) ihn hinzugefügt und (4) eine sinnvolle Mini-Aufgabe durchgeführt. Beispiel: der GitHub-MCP + "liste die offenen Issues im Repo X auf und schlag vor, welches für einen Anfänger am leichtesten ist". **Der Sicherheitscheck, den du hättest machen sollen:** Ist der Server offiziell/bekannt? Was kann er alles (nur lesen oder auch schreiben)?

Lösung Ü4.7

Referenzantwort zum Theorieteil (vergleiche deine): Ein API-Aufruf aus dem Code ist Teil der ANWENDUNG — er läuft bei jeder Nutzung, für jeden Nutzer (das Wetter MUSS ein API-Aufruf aus dem Code sein, denn die Nutzer der App haben kein Claude). Ein MCP-Server ist ein Werkzeug für CLAUDE während der ENTWICKLUNG — er gibt ihm Augen/Hände in externe Systeme, während er dir hilft (die Datenbank während der Entwicklung zu lesen ist ideal für MCP, denn die Nutzer brauchen das nicht, und dir und Claude beschleunigt es die Arbeit). Faustregel: **was die App braucht → Code; was Claude braucht, um dir zu helfen → MCP. Praktischer Teil — Kriterien:**

- ☐ Claude ist über den Browser-MCP mindestens 2 Szenarien durchgegangen (Erfassen, Ansehen)
- ☐ Hat den Datenbankstand nach den Szenarien über den SQLite-MCP bestätigt
- ☐ Der Bericht enthält: was funktioniert / was nicht / was seltsam ist
- ☐ Du hast mindestens einen Befund aus dem Bericht persönlich geprüft (Vertrauen wird geprüft!)

Modul 5 — Lösungen

Lösung Ü5.1

Referenz-Prompt:

Schick einen Explore-Subagenten, der untersucht, wie dieses Projekt mit Fehlern umgeht (Exceptions/Error Handling): wo die zentrale Logik liegt, welche Fehlertypen es definiert, wie ein Nutzer der Bibliothek Fehler erhält. Er soll einen kurzen Bericht mit den Pfaden der Schlüsseldateien zurückbringen.

Erwartetes Ergebnis: ein Bericht von 10–20 Zeilen mit konkreten Pfaden; deine Sitzung ist nicht "schwer" geworden (der Agent hat in seinem eigenen Kontext gelesen). **Häufiger Fehler:** eine zu schwammige Aufgabe für den Agenten ("untersuch das Projekt") — ein Agent ist am BESTEN mit einer engen, konkreten Frage. Ein Subagent sieht euer Gespräch nicht — alles, was er braucht, muss in der Aufgabe stehen.

Lösung Ü5.2

Referenz-Prompt:

Starte 3 Explore-Subagenten PARALLEL: (1) wie in diesem Projekt Tests geschrieben und ausgeführt werden, (2) wie die Konfiguration organisiert ist, (3) welche externen Abhängigkeiten es nutzt und wozu jede dient. Wenn sie fertig sind, führe die Ergebnisse zu einer Projektübersicht für ein neues Teammitglied zusammen.

Erwartetes Ergebnis: drei Agenten arbeiten gleichzeitig; du bekommst eine zusammengeführte Übersicht. Bemerke, wie viel Kontext es gefressen hätte, all das in deiner Sitzung zu lesen.

Lösung Ü5.3

Erwartete Beobachtung: direktes Lesen = schneller bei KLEINEN Fragen (kein "Overhead" des Agentenstarts), aber dein Kontext füllt sich sichtbar; ein Subagent = lohnt sich, wenn die Antwort das Lesen vieler Dateien erfordert und du nur das FAZIT brauchst, nicht den Rohinhalt. **Deine Faustregel (vergleiche):** eine Frage zu 1–2 bekannten Dateien → direkt; "wie ist X im ganzen Projekt organisiert" → ein Subagent; mehrere unabhängige Fragen → parallele Subagenten.

Lösung Ü5.4

Komplettes Beispiel `.claude/agents/rezensent.md` :

```
---  
name: rezensent  
description: Read-only Code-Review anhand einer Checkliste. Nutzen,  
    wenn ein Review verlangt wird. Gibt eine Liste von Befunden zurück,  
    ändert nichts.  
tools: Read, Grep, Glob  
---
```

Du bist ein strenger, aber fairer Code-Rezensent. Du änderst NIE
Dateien – du liest nur und berichtest.

Prüfe den Code anhand dieser Checkliste:

1. Lesbarkeit: Sagen Variablen-/Funktionsnamen, was sie tun?
2. Wiederholung: derselbe Code an mehreren Stellen, der hätte ausgelagert werden sollen?
3. Offensichtliche Bugs: Vergleiche, Schleifengrenzen, null/undefined, Randfälle.
4. Fehlerbehandlung: Was passiert, wenn etwas schiefgeht?
5. Toter Code: Funktionen/Variablen, die nirgends genutzt werden.

Berichtsformat – für jeden Befund:

- [pfad:zeile] Kurze Problembeschreibung → konkreter Korrekturvorschlag
- Schweregrad: BUG / RISIKO / STIL

Am Ende: 2 Sätze Gesamteindruck. Melde keine Stil-Kleinigkeiten,
die das Verständnis nicht erschweren. Höchstens die 10 wichtigsten
Befunde.

Warum so: `tools: Read, Grep, Glob` verhindert PHYSISCH, dass er ändert (read-only ist eine Beschränkung, keine Bitte); die `description` sagt, WANN er genutzt wird (wichtig für automatisches Aufrufen); das Berichtsformat ist fixiert, damit die Ausgabe von Lauf zu Lauf vergleichbar ist; das Befundlimit verhindert Zuschütten.

Lösung Ü5.5

Vorgehen: Rufe im Quiz-Ordner den Rezensenten auf ("lass den Agenten rezensent über das ganze Projekt laufen"). Erwarte 5–10 Befunde; typisch für ein schnell gebautes Quiz: duplizierter DOM-Code, Magic Numbers (die 15 Sekunden an 3 Stellen), Umgang mit leerem Spielernamen. **Kriterium:** Zu jedem Befund hast du DEINE eigene Entscheidung getroffen (beheben/ignorieren + warum). Der Rezensent (und KI überhaupt) schlägt vor — du entscheidest.

Lösung Ü5.6

Erfolgskriterien:

- ☐ Die drei Fragen sind DEINE und eng genug, dass ein Agent konkret antworten kann

- ☐ Jeder Befund im Bericht hat Dateipfade (ohne Pfade = Nacherzählung, keine Analyse)
- ☐ Deine Einschätzung "am schwersten zu ändern" zitiert konkrete Befunde (z.B. "Logik X ist über N Dateien verteilt, jede Änderung berührt alle")

Wenn der Bericht keine Pfade hat: Schärfe die Aufgabe des Agenten nach — verlange die Pfade explizit. Ein Agent tut genau das, was in der Aufgabe steht.

Modul 6 — Lösungen

Lösung Ü6.1

Referenz-Prompt:

Initialisiere ein Git-Repo. Erstelle eine .gitignore passend für [Projekttyp] und eine CLAUDE.md mit den Grundregeln des Projekts ([deine Regeln]). Implementiere dann [erstes Feature] und schlag einen Commit vor – aber zeig mir die Nachricht, bevor du committest.

Was du an der Commit-Nachricht prüfst: dass sie BESCHREIBT, WAS die Änderung tut (nicht "Änderungen" oder "Update"), kurz und konkret. Schlecht: "Dateien hinzugefügt". Gut: "Fügt Aufgabenerfassung und -auflistung mit JSON-Speicherung hinzu".

Lösung Ü6.2

Referenz-Ablauf: Erstelle einen Branch feature-X und implementiere: [Beschreibung] → wenn fertig → /code-review → du liest die Befunde → Behebe Befunde 1 und 3, Befund 2 ist Absicht → Commit. **Der Punkt:** Ein Review VOR dem Commit heißt, dass in der Historie eine saubere Version bleibt. **Häufiger Fehler:** blind ALLE Befunde beheben — das Review-Tool kennt deine Absichten nicht; du beurteilst, welche Befunde berechtigt sind.

Lösung Ü6.3

Referenz-Prompt: Veröffentliche dieses Repo auf meinem GitHub-Konto als öffentliches Repo [name], pushe main und den Branch, und öffne einen Pull Request vom Branch auf main mit einer Beschreibung, was das Feature tut und wie es getestet wurde. **Erwartetes Ergebnis:** das Repo auf GitHub; ein PR mit sinnvoller Beschreibung; nach der Durchsicht im Browser hast du ihn gemergt (geht auch über Claude: "merge den PR"). **Eine oft vergessene Voraussetzung:** gh auth login muss vorher erledigt sein.

Lösung Ü6.4

Beispiel .claude/settings.json mit einem Hook (Python-Projekt — Syntaxprüfung nach jeder Dateiänderung):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "python -m py_compile \"${CLAUDE_FILE_PATHS}\" && echo Syntax OK"
          }
        ]
      }
    ]
  }
}
```

(Die genauen Ereignisnamen und Variablen prüfst du in der Doku — `/help`, oder bitte Claude "richte mir einen Hook ein, der nach jeder Änderung einer .py-Datei die Syntax prüft" und lass ihn `settings.json` selbst schreiben — und LIES dann, was er geschrieben hat.) **Test:** Verlange eine Codeänderung → du siehst in der Sitzung, dass der Hook ausgelöst hat. Bitte ihn dann absichtlich, einen Syntaxfehler einzubauen — der Hook sollte ihn fangen. **Antwort auf die Frage aus der Übung:** In Hooks gehören Regeln, die gelten MÜSSEN (Formatierung, Tests, kein Commit auf main...) — CLAUDE.md ist eine Anleitung, die gelesen wird, ein Hook ist ein Tor, an dem man nicht vorbeikommt.

Lösung Ü6.5

Komplettes Beispiel `.claude/commands/liefern.md`:

description: Prüfe, committe und pushe die aktuellen Änderungen

Tu Folgendes, der Reihe nach, und halte an, wenn ein Schritt fehlschlägt:

1. Führe die Prüfungen des Projekts aus, falls vorhanden (Tests/Linter); falls nicht, überspringe mit einem Hinweis.
2. Sieh dir alle aktuellen Änderungen an (git status und git diff).
3. Verfasse eine kurze Commit-Nachricht, die beschreibt, WAS die Änderungen tun.
4. Zeig mir die Nachricht und die Dateiliste, dann warte auf meine Bestätigung.
5. Nach der Bestätigung: Commit und Push auf den aktuellen Branch.

Argument (optional): \$ARGUMENTS – falls übergeben, nutze es als Grundlage für die Commit-Nachricht.

Verwendung: /liefern oder /liefern Timer-Korrektur. **Warum es Schritt 4 gibt:** Ein Befehl, der committet, ohne es dir zu zeigen = Automatisierung ohne Kontrolle; lass bei unumkehrbaren Schritten immer einen Kontrollpunkt.

Lösung Ü6.6

Beispielskript (tagesbericht.sh / analog für PowerShell):

```
#!/bin/sh
claude -p "Sichte das Projekt: liste alle TODO- und FIXME-Kommentare
mit Pfaden auf, gruppieren sie nach Thema und schlag die Reihenfolge
der Abarbeitung vor, mit Begründung. Ändere nichts." > tagesbericht.txt
```

Erwartetes Ergebnis: von Hand ausgeführt liefert es einen brauchbaren Bericht in einer Datei. **Die Idee weitergedacht:** so ein Skript im CI oder Scheduler = Claude als Teil der Maschinerie, nicht nur als Gesprächspartner. Hinweis: Denk im Headless-Modus gut über Berechtigungen nach — fang mit Aufgaben an, die nur LESEN.

Lösung Ü6.7

Erfolgskriterien der Abschlussprüfung — über die Checkliste der Aufgabe hinaus, qualitativ:

- ☐ Du kannst das Repo jemandem zeigen und jede Entscheidung erklären (warum Plan-Modus genau dort, warum dieser MCP, was der Hook garantiert)
- ☐ CLAUDE.md ist so, dass ein neuer Mensch (oder eine neue Sitzung) die Arbeit ohne deine mündlichen Erklärungen fortsetzen würde — TESTE ES: neue Sitzung + Anfrage für eine kleine

Verbesserung

- ☐ Die Commit-Historie liest sich wie die Entstehungsgeschichte des Projekts
- ☐ Es gibt mindestens einen Moment, in dem du etwas von Claude Vorgeschlagenes mit Begründung ABGELEHNT hast — das Zeichen, dass du führst, nicht das Tool

Wenn alles bestanden ist: Glückwunsch — du bist den Weg von "Ich tippe Prompts" zu "Ich führe einen Agenten durch den ganzen Software-Lebenszyklus" gegangen. Das war das Ziel des Kurses.

Dieser Kurs darf frei geteilt werden. Er wurde anhand der öffentlich verfügbaren Claude-Code-Dokumentation geschrieben (<https://code.claude.com/docs>); Befehle und Namen können sich mit der Zeit ändern — wenn etwas nicht so funktioniert, wie der Kurs sagt, frag das Tool selbst (`/help`) oder schau in die Dokumentation.